



Documentação - Análise de Dados - E-commerce Olist (DataGirls)

1. Equipe e Documentação:

Projeto desenvolvido individualmente por **Carla Bruckmann**.

[Repositório GitHub](#)

2. Objetivo:

Aplicar conceitos de análise de dados em um cenário real de e-commerce, explorando métricas de vendas, comportamento do cliente, logística e satisfação, utilizando SQL no BigQuery.

3. Ferramentas e Tecnologias:

- BigQuery (SQL).
- Notion (Documentação).
- Git/GitHub (Repositório).

4. Fonte dos Dados:

[Database Olist E-commerce](#)

▼ Descrição de tabelas

- **Tabela de Clientes (`olist_customers_dataset`)**

Dados de quem comprou.

`customer_id`: Chave do cliente atrelada ao pedido. Importante: A cada nova compra, o sistema gera um novo customer_id. Ele serve para cruzar com a tabela de Pedidos.

`customer_unique_id`: O "CPF" real do cliente. Coluna que possibilita saber se o cliente fez mais de uma compra na loja (recompra).

`customer_zip_code_prefix`: Os 5 primeiros dígitos do CEP.

`customer_city`: Cidade do cliente.

`customer_state`: Estado do cliente.

- **Tabela de Pedidos (`olist_orders_dataset`)**

A "espinha dorsal" da base. Conecta o cliente, o pagamento e os produtos. Possibilita análises de logística e tempo de entrega.

`order_id`: Chave única do pedido.

`customer_id`: Chave para cruzar com a tabela de clientes (localidade da pessoa que fez o pedido).

`order_status`: Status do pedido (ex: delivered, shipped, canceled).

`order_purchase_timestamp`: Data e hora exata da compra.

`order_approved_at`: Data e hora que o pagamento foi aprovado.

`order_delivered_carrier_date`: Quando o pedido foi entregue à transportadora.

`order_delivered_customer_date`: Data real em que chegou na casa do cliente. (Subtrair essa data da data de compra resulta o tempo de entrega).

`order_estimated_delivery_date`: Data prometida para entrega. (Cruzar com a data real apresenta se houve atraso).

- **Tabela de Itens do Pedido (`olist_order_items_dataset`)**

Mostra "o que" tem dentro de cada caixinha de pedido. (Se a pessoa comprou 3 cadeiras e 1 mesa no mesmo pedido, haverá 4 linhas).

`order_id`: Chave para cruzar com a tabela de Pedidos.

`order_item_id`: Número sequencial do item dentro daquele pedido (ex: item 1, item 2).

`product_id`: Chave única do produto (cruza com a tabela de Produtos).

`seller_id`: Chave do lojista parceiro que vendeu aquele item.

`price`: Preço do produto.

`freight_value`: Valor do frete cobrado por aquele item específico.

- **Tabela de Pagamentos (`olist_order_payments_dataset`)**

Como o dinheiro entrou.

`order_id`: Chave para cruzar com a tabela de Pedidos.

`payment_sequential`: Se o cliente pagou com 2 cartões diferentes, marca 1 e 2.

`payment_type`: Método de pagamento (ex: credit_card, boleto, voucher).

`payment_installments`: Quantidade de parcelas.

`payment_value`: Valor total daquela transação.

- **Tabela de Avaliações (`olist_order_reviews_dataset`)**

A voz do cliente. Permite cruzar a "nota" com "atraso na entrega" ou "preço do frete".

`review_id`: Chave única da avaliação.

`order_id`: Chave do pedido avaliado.

`review_score`: A nota que o cliente deu, de 1 a 5 estrelas. (KPI principal de satisfação).

`review_comment_title`: Título do comentário.

`review_comment_message`: O texto do comentário. (Muitos estarão vazios, bom para ensinar tratamento de nulos).

`review_creation_date`: Quando o questionário de satisfação foi enviado.

`review_answer_timestamp`: Quando o cliente respondeu.

- **Tabela de Produtos (`olist_products_dataset`)**

Características físicas do que foi vendido.

`product_id`: Chave única do produto.

`product_category_name`: Nome da categoria (ex: beleza_saude,cama_mesa_banho).

`product_weight_g`: Peso em gramas.

`product_length_cm / height_cm / width_cm`: Dimensões do pacote (Comprimento, altura, largura).

- **Tabelas de Suporte**

`olist_sellers_dataset`: Dados dos lojistas (cidade, estado). Igual à de clientes, mas para o vendedor.

`olist_geolocation_dataset`: Uma base de dados gigante relacionando CEPs a latitudes e longitudes.

`product_category_name_translation`: "De/Para" traduzindo as categorias do Português para o Inglês.

- **Tabela Analítica (`orders_enriched`):**

Tabela consolidada no nível de pedido (`order_id`), contendo métricas agregadas e dados enriquecidos do cliente.

Servir como base para análises de comportamento, performance e satisfação do cliente.

Métricas incluídas:

- `valor_total` → soma dos itens do pedido
- `frete_total` → soma do frete
- `quantidade_itens` → total de itens no pedido
- `review_score` → média de avaliação (quando disponível)

Observação:

Pedidos sem avaliação permanecem na base com `review_score = NULL`, devido ao uso de LEFT JOIN.

5. Pré-processamento:

▼ Importação de dados

Dados originalmente fornecidos em GoogleDrive, baixados localmente e importados para o BigQuery.

- **BigQuery:**

- Criação de **Projeto** `Olist-DataGirls` (ID: `sql-olist-datagirls` ;
- Criação de **Conjunto de Dados** `Database` ;
- Importação de **Tabelas**:
 - `olist_customers_dataset`
 - `olist_geolocation_dataset`
 - `olist_order_items_dataset`
 - `olist_order_payments_dataset`
 - `olist_orders_dataset`
 - `olist_products_dataset`

- `olist_sellers_dataset`
- `product_category_name_translation`
- `olist_order_reviews_dataset` → Importante: Tabela importada com aviso de erros. **(Necessária ativação de : Permitir quebras de linha entre aspas).**

▼ Camada Analytics

- Criação do dataset `analytics`
- Construção da tabela `orders_enriched` com métricas agregadas no nível de pedido

6. Atividades:

Aula - 23 de março de 2026

▼ 1 - Pedidos realizados:

Liste os 10 primeiros pedidos com:

```
order_id
order_status
order_purchase_timestamp
```

Resposta:

```
SELECT
  order_id,
  order_status,
  order_purchase_timestamp
FROM `sql-olist-datagirls.Database.olist_orders_dataset`
ORDER BY order_purchase_timestamp ASC --garante 10 primeir
os pedidos cronológicamente
LIMIT 10; -- limita a 10
```

▼ 2 - Pedidos entregues:

Liste todos os pedidos com status "delivered"

Mostre:

```
order_id
```

```
order_purchase_timestamp  
order_delivered_customer_date
```

Resposta:

```
SELECT  
order_id,  
order_purchase_timestamp,  
order_delivered_customer_date  
FROM `sql-olist-datagirls.Database.olist_orders_dataset`  
WHERE order_status = "delivered"; --condicional
```

▼ 3 - Liste clientes do estado de SP:

Mostre:

```
customer_unique_id  
customer_city
```

Resposta:

```
SELECT  
customer_unique_id,  
customer_city  
FROM `sql-olist-datagirls.Database.olist_customers_dataset`  
  
WHERE customer_state = "SP";
```

▼ 4 - Itens caros:

Liste os itens com preço maior que 500

Mostre:

```
order_id  
product_id  
price
```

Resposta:

```
SELECT  
order_id,  
product_id,  
price
```

```
FROM `sql-olist-datagirls.Database.olist_order_items_datas`  
et`  
WHERE price > 500;
```

▼ 5 - Quantidade de pedidos por status:

Conte quantos pedidos existem por `order_status`

Resposta:

```
SELECT  
order_status,  
COUNT(order_id) AS total_pedido_status --conta e nomeia  
FROM `sql-olist-datagirls.Database.olist_orders_dataset`  
GROUP BY order_status;
```

▼ 6 - Receita total por tipo de pagamento:

Some o payment_value por `payment_type`

Resposta:

```
SELECT  
payment_type,  
SUM(payment_value) AS valor_total_pagamento  
FROM `sql-olist-datagirls.Database.olist_order_payments_dataset`  
GROUP BY payment_type;
```

▼ 7 - Ticket médio por pedido:

Calcule o valor médio de pagamento por pedido

Resposta:

```
SELECT  
    AVG(valor_pedido) AS ticket_medio --média subquery  
FROM (  
    SELECT
```

```

        order_id,
        SUM(payment_value) AS valor_pedido --soma os valores p
or id
    FROM `sql-olist-datagirls.Database.olist_order_payments_
dataset`
    GROUP BY order_id
);

```

▼ 8 - Pedidos com cidade do cliente:

Junte pedidos com clientes

Mostre:

```

order_id
customer_city
customer_state

```

Resposta:

```

SELECT
    o.order_id,
    c.customer_city, --seleciona das tabelas → aliases para
simplificar a leitura
    c.customer_state
FROM `sql-olist-datagirls.Database.olist_orders_dataset` A
S o
JOIN `sql-olist-datagirls.Database.olist_customers_dataset`
` AS c --junta as tabelas
    ON o.customer_id = c.customer_id; -- relação entre pedid
o e cliente

```

▼ 9 - Produtos vendidos por pedido:

Junte pedidos com itens

Mostre:

```

order_id
product_id
price

```

Resposta:


```

SELECT
    o.order_id,
    p.product_id, --seleciona das tabelas → aliases para simplificar a leitura
    p.price
FROM `sql-olist-datagirls.Database.olist_orders_dataset` AS o
JOIN `sql-olist-datagirls.Database.olist_order_items_dataset` AS p --junta as tabelas
    ON o.order_id = p.order_id; -- relação entre pedido e produto

```

▼ 10 - Tempo de entrega:

Calcule o tempo de entrega (em dias)

Mostre:

```

order_id
data da compra
data de entrega
tempo de entrega

```

Resposta:

```

SELECT
    order_id,
    order_purchase_timestamp,
    order_delivered_customer_date,
    TIMESTAMP_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY) AS tempo_entrega_dias --TIMESTAMP_DIFF(data_final, data_inicial, unidade)
FROM `sql-olist-datagirls.Database.olist_orders_dataset`
WHERE order_status = 'delivered';

```

Tarefa - 24 de março de 2026

▼ 1 - Pedidos atrasados:

Liste pedidos onde:
data real > data estimada

Resposta:

```
SELECT
    order_id,
    DATE(order_delivered_customer_date) AS data_entrega, --
    -data(sem considerar a hora)
    DATE(order_estimated_delivery_date) AS data_estimada,
    DATE_DIFF(
        DATE(order_delivered_customer_date),
        DATE(order_estimated_delivery_date),
        DAY
    ) AS dias_atraso
FROM `sql-olist-datagirls.Database.olist_orders_dataset`
WHERE order_delivered_customer_date IS NOT NULL
AND order_estimated_delivery_date IS NOT NULL
AND DATE(order_delivered_customer_date) > DATE(order_estimated_delivery_date)
ORDER BY dias_atraso DESC;
```

▼ 2 - Clientes recorrentes:

Encontre clientes (`customer_unique_id`) que fizeram mais de 1 pedido

Anotação:

Cláusula	Quando usa	O que filtra
<code>WHERE</code>	antes do <code>GROUP BY</code>	linhas
<code>HAVING</code>	depois do <code>GROUP BY</code>	grupos

Resposta:

```
SELECT
    c.customer_unique_id, --seleciona das tabelas → alias
    s para simplificar a leitura
```

```

COUNT(o.order_id) AS pedidos --conta os pedidos
FROM `sql-olist-datagirls.Database.olist_orders_dataset` A
S o
JOIN `sql-olist-datagirls.Database.olist_customers_dataset`
` AS c --junta as tabelas
ON o.customer_id = c.customer_id -- relação entre pedi
do e cliente
GROUP BY c.customer_unique_id
HAVING COUNT (o.order_id) >1;--quantidade de pedidos >1

```

▼ 3 - Pedidos por estado:

Conte pedidos por estado do cliente

Resposta:

```

SELECT
    c.customer_state,
    COUNT(o.order_id) AS pedidos --conta os pedidos
FROM `sql-olist-datagirls.Database.olist_orders_dataset` A
S o
JOIN `sql-olist-datagirls.Database.olist_customers_dataset`
` AS c --junta as tabelas
ON o.customer_id = c.customer_id -- relação entre pedi
do e cliente
GROUP BY c.customer_state
ORDER BY pedidos DESC;

```

▼ 4 - Avaliação média:

Calcule a média de `review_score`

Resposta:

```

SELECT
    AVG(review_score) AS media_avaliacao

```

```
FROM `sql-olist-datagirls.Database.olist_order_reviews_dataset`;
```

▼ 5 - Receita por estado:

Some o valor total de pagamentos por estado

Anotações:

- pagamentos (`order_payments`) order_id
- pedidos (`orders`) order_id e customer_id
- clientes (`customers`) customer_id

Resposta:

```
SELECT
  c.customer_state,
  SUM(payment_value) AS receita_total
FROM `sql-olist-datagirls.Database.olist_order_payments_dataset` AS p
JOIN `sql-olist-datagirls.Database.olist_orders_dataset` AS o
  ON o.order_id = p.order_id
JOIN `sql-olist-datagirls.Database.olist_customers_dataset` AS c
  ON o.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY receita_total DESC;
```

▼ 6 - Avaliação por status:

Relacione `review_score` médio por `order_status`

Anotações:

- `review_score` - `olist_order_reviews_dataset` - order_id
- `order_status` - `olist_orders_dataset` - order_id

Resposta:

```
SELECT
    o.order_status,
    AVG(r.review_score) AS media_avaliacao
FROM `sql-olist-datagirls.Database.olist_order_reviews_dataset` as r
JOIN `sql-olist-datagirls.Database.olist_orders_dataset` AS o
    ON r.order_id = o.order_id
GROUP BY o.order_status
ORDER BY media_avaliacao DESC;
```

▼ 7 - Categoria mais vendida:

Junte:

itens + produtos

Descubra qual categoria tem mais vendas

Anotações:

- itens (`order_item_id`) - `olist_order_items_dataset` `product_id`
- produtos (`product_category_name`)- `olist_products_dataset` `product_id`

Resposta:

Categoria com mais itens vendidos:

```
SELECT
    p.product_category_name,
    COUNT(o.order_item_id) AS total_itens --conta os itens
    pedidos
FROM `sql-olist-datagirls.Database.olist_order_items_dataset` AS o
JOIN `sql-olist-datagirls.Database.olist_products_dataset`
    AS p --junta as tabelas
    ON o.product_id = p.product_id -- relação
```

```
GROUP BY p.product_category_name
ORDER BY total_itens DESC;
```

▼ 8 - Frete médio por categoria:

Calcule o frete médio por categoria de produto

Anotações:

- frete (`freight_value`)- `olist_order_items_dataset` `product_id`
- produtos (`product_category_name`)- `olist_products_dataset` `product_id`

Resposta:

```
SELECT
    p.product_category_name,
    AVG(o.freight_value) AS media_frete
FROM `sql-olist-datagirls.Database.olist_order_items_dataset` AS o
JOIN `sql-olist-datagirls.Database.olist_products_dataset`
AS p --junta as tabelas
    ON o.product_id = p.product_id -- relação
GROUP BY p.product_category_name
ORDER BY media_frete DESC;
```

▼ 9 - Criar base analítica:

Crie uma tabela com:

```
order_id
customer_state
data_compra
valor_total (soma dos itens)
frete_total
quantidade_itens
review_score
```

(DICA: JOIN de várias tabelas + GROUP BY)

Anotações:

- `orders` → `order_id` , `customer_id` , `order_purchase_timestamp`
- `customers` → `customer_state`

- `order_items` → `price`, `freight_value`, `order_item_id`
- `order_reviews` → `review_score`

LEFT JOIN

- Mantém **todos os pedidos** da tabela `orders`
- Só traz `review_score` quando existe correspondência
- Quando **não existe review**, os campos da tabela `r` ficam `NULL`

Resposta:

```
--orders_enriched
SELECT
    o.order_id,
    c.customer_state,
    DATE(o.order_purchase_timestamp) AS data_compra,
    SUM(oi.price) AS valor_total, --valor total
    SUM(oi.freight_value) AS frete_total, -- frete
    COUNT(oi.order_item_id) AS quantidade_itens, --quantid
ade itens
    AVG(r.review_score) AS review_score --media score (cas
o tenha mais de 1 review por pedido)
FROM `sql-olist-datagirls.Database.olist_orders_dataset` A
S o
JOIN `sql-olist-datagirls.Database.olist_customers_dataset`
` AS c
    ON o.customer_id = c.customer_id
JOIN `sql-olist-datagirls.Database.olist_order_items_datas
et` AS oi
    ON o.order_id = oi.order_id
LEFT JOIN `sql-olist-datagirls.Database.olist_order_review
s_dataset` AS r
    ON o.order_id = r.order_id
GROUP BY
    o.order_id,
    c.customer_state,
```

```
data_compra  
ORDER BY data_compra DESC;
```

▼ 10 - Exportar resultado:

No BigQuery:

Criação de **Conjunto de Dados** `analytics` ;

- Importação de **Tabela**:

- `orders_enriched`

Formulário → comprovante de entrega:

Exercícios semana 2 - SQL

Sua resposta foi registrada.

[Enviar outra resposta](#)